



Projet	Annuaire Active Directory (LDAP)
Stack	React 18 + TypeScript + Vite + Tailwind CSS
Backend	Symfony + API Platform + JWT
Auteur	Irina
Date	09/04/2026
Version	1.0.0

Table des matières

Table des matières	2
1. Vue d'ensemble	4
1.1 Objectifs	4
1.2 Architecture générale	4
2. Stack technique	5
2.1 Frontend.....	5
2.2 Backend	5
3. Structure du projet.....	6
3.1 Arborescence frontend	6
4. Authentification & Sécurité.....	7
4.1 Flux complet.....	7
4.2 Structure du JWT.....	7
4.3 Stockage local	7
4.4 Sécurité.....	7
5. Composants principaux	8
5.1 AuthContext.....	8
5.2 ProtectedLayout.....	8
5.3 Header	8
5.4 LdapList	8
5.5 SearchBar	8
5.6 AccessDenied.....	9
6. API & Client HTTP	10
6.1 Configuration Axios (client.ts)	10
6.2 Intercepteur de requête	10
6.3 Intercepteur de réponse	10
6.4 Routes API utilisées.....	10
7. Variables d'environnement	11
7.1 Frontend (.env)	11
7.2 Backend (.env)	11
8. Routing.....	12
8.1 Configuration des routes	12
8.2 Chargement paresseux (Lazy Loading).....	12
9. Charte graphique	13
9.1 Palette de couleurs	13
9.2 Typographie	13
9.3 Composants UI réutilisables	13
10. Problèmes connus & solutions	14

10.1 CORS.....	14
10.2 Race condition token.....	14
10.3 userRole.toUpperCase() crash	14
11. Lancement du projet.....	15
11.1 Prérequis.....	15
11.2 Frontend.....	15
11.3 Backend	15

1. Vue d'ensemble

L'annuaire LDAP est une application web interne permettant aux employés de HFF de consulter le répertoire des utilisateurs de l'Active Directory. L'accès est sécurisé via un système JWT géré par l'intranet PHP existant.

1.1 Objectifs

- Afficher l'annuaire complet des employés (nom, fonction, email, téléphone, localisation)
- Permettre la recherche et le filtrage en temps réel
- Exporter les données au format Excel
- S'intégrer de manière transparente avec l'intranet PHP existant
- Gérer les rôles utilisateur (ADMIN / COLLABORATEUR)

1.2 Architecture générale

Flux d'authentification
PHP Login → Génération JWT → Redirection React (?token=xxx) → Décodage JWT → Stockage localStorage → Appel API avec Bearer token

Frontend	React 18 + TypeScript, servi via Vite
Auth	JWT généré par l'intranet PHP, décodé côté React
LDAP	Active Directory interrogé par Symfony
Communication	Axios avec intercepteurs (Bearer token + refresh)

2. Stack technique

2.1 Frontend

Fichier	Type	Rôle
React 18	Framework	Interface utilisateur
TypeScript	Langage	Typage statique
Vite	Bundler	Développement et build
Tailwind CSS	CSS	Styles utilitaires
Axios	HTTP	Appels API avec intercepteurs
jwt-decode	Auth	Décodage du token JWT
React Router v6	Routing	Navigation et routes protégées
ExcelJS	Export	Génération de fichiers .xlsx
Lucide React	UI	Icônes

2.2 Backend

Fichier	Type	Rôle
Symfony	Framework	API REST
API Platform	Bundle	Exposition automatique des entités
LexikJWT	Auth	Vérification des tokens JWT
NelmioCORS	Sécurité	Gestion des headers CORS
LDAP Component	Directory	Connexion Active Directory

3. Structure du projet

3.1 Arborescence frontend

src/	
App.tsx	← Point d'entrée React
main.tsx	← Montage de l'app
index.css	← Styles globaux + variables CSS
components/	
layout/	
Header.tsx	← En-tête (logo, dropdown utilisateur)
Footer.tsx	← Pied de page (documentation, date)
ProtectedLayout.tsx	← Layout protégé (vérification auth)
ui/	
Breadcrumb.tsx	← Fil d'ariane (Accueil > Annuaire)
SearchBar.tsx	← Formulaire de recherche 4 champs
Button.tsx	← Composant bouton réutilisable
Loader.tsx	← Spinner de chargement
pages/Ldap/	
LdapList.tsx	← Tableau annuaire + pagination + export
contexts/	
AuthContext.tsx	← Contexte auth (user, logout, revalidate)
lib/api/	
client.ts	← Instance Axios + intercepteurs
auth.api.ts	← logout(), checkSession()
ldap.api.ts	← list() — GET /directory
pages/	
ldap/LdapPage.tsx	← Page principale annuaire
AccessDenied.tsx	← Page 403
router/	
index.tsx	← Routes (protégées + publiques)
types/	
user.type.ts	← Interface User + UserRole

4. Authentification & Sécurité

4.1 Flux complet

L'authentification est entièrement gérée par l'intranet PHP. React est un consommateur passif du token JWT.

Étape 1	L'utilisateur se connecte sur l'intranet PHP
Étape 2	PHP vérifie les credentials et génère un JWT
Étape 3	PHP redirige vers React : <code>http://app:5173?token=eyJ...</code>
Étape 4	App.tsx (hors composant) decode le JWT et stocke les données
Étape 5	localStorage reçoit : <code>accessToken</code> + <code>user</code> (<code>fullname</code> , <code>role</code> , <code>url_logout</code>)
Étape 6	Header affiche le nom et le rôle de l'utilisateur
Étape 7	LdapList appelle <code>/api/directory</code> avec <code>Authorization: Bearer xxx</code>

4.2 Structure du JWT

Le token JWT envoyé par PHP contient le payload suivant :

<code>fullname</code>	Nom complet de l'utilisateur (fallback: <code>name</code> , <code>displayname</code>)
<code>email / mail</code>	Adresse email
<code>role</code>	Valeur numérique : 1 = ADMIN, 0 = COLLABORATEUR
<code>roles</code>	Tableau de rôles numériques (alternative à <code>role</code>)
<code>url_logout</code>	URL de déconnexion de l'intranet PHP

4.3 Stockage local

<code>accessToken</code>	Le JWT brut, utilisé dans le header <code>Authorization</code>
<code>user</code>	Objet JSON : <code>{ fullname, email, role, url_logout }</code>

4.4 Sécurité

- L'URL est nettoyée après récupération du token (`window.history.replaceState`)
- Le token est sauvegardé AVANT le rendu des composants (hors `useEffect`)
- `ProtectedLayout` redirige vers `/access-denied` si non authentifié
- `LdapList` affiche un message d'erreur en cas de 401 ou 403 sans déconnecter
- Le `logout` nettoie le `localStorage` et redirige vers l'intranet PHP

5. Composants principaux

5.1 AuthContext

Contexte React global qui centralise l'état d'authentification. Exposé via le hook `useAuth()`.

user	Objet User ou null
isAuthenticated	Boolean : true si user + accessToken présents
isLoading	Boolean : true pendant la vérification initiale
logout()	Nettoie localStorage + redirige vers PHP
removeAuth()	Nettoie localStorage sans redirection
revalidateAuth()	Vérifie la session via <code>/api/session</code> (si dispo)

5.2 ProtectedLayout

Composant wrapper qui protège toutes les routes enfants. Vérifie l'authentification et affiche un loader pendant la vérification.

- Affiche un Loader pendant `isLoading`
- Redirige vers `/access-denied` si `!isAuthenticated`
- Rend Header + Breadcrumb + `<Outlet />` + Footer si authentifié
- Revalide l'auth à chaque changement de route

5.3 Header

En-tête fixe (sticky) affichant le logo et un dropdown utilisateur avec nom, badge de rôle et bouton de déconnexion.

- Lit `user_data` depuis AuthContext (via `useAuth`)
- Badge rouge ADMIN / badge bleu COLLABORATEUR selon `user.role`
- Détection multi-onglets via `StorageEvent` (déconnexion synchronisée)
- Fermeture du dropdown via overlay ou touche Échap

5.4 LdapList

Composant principal affichant le tableau de l'annuaire avec pagination (50 entrées/page) et export Excel.

- Appel API via `LdapApi.list()` (GET `/api/directory` + Bearer token)
- Filtrage côté client sur 4 champs : nom, localisation, email, fonction
- Pagination intelligente avec ellipsis (...)
- Export Excel via `ExcelJS` avec en-tête coloré (`#1F2937`)
- Gestion douce des erreurs : 401, 403, erreur réseau

5.5 SearchBar

Formulaire de recherche collapsible avec 4 champs de filtrage en temps réel.

- Filtrage déclenché à chaque frappe (onChange)
- Bouton EFFACER réinitialise tous les champs
- Bouton RECHERCHE déclenche le filtrage manuellement
- Collapsible via animation Tailwind (max-h transition)

5.6 AccessDenied

Page publique (hors ProtectedLayout) affichée quand l'utilisateur n'est pas authentifié ou accède sans token.

- Route : /access-denied
- Bouton "Retourner à l'accueil" redirige vers VITE_API_URL_LOGOUT
- Accessible sans token (pas de vérification d'auth)

6. API & Client HTTP

6.1 Configuration Axios (client.ts)

baseURL	VITE_API_URL (défini dans .env)
timeout	10 000 ms
withCredentials	true (pour les cookies httpOnly)
Content-Type	application/json

6.2 Intercepteur de requête

Ajoute automatiquement le header Authorization: Bearer <token> à chaque requête si un accessToken est présent dans le localStorage.

6.3 Intercepteur de réponse

En cas d'erreur 401, tente un refresh automatique via POST /auth/refresh-token (cookie httpOnly). Si le refresh échoue, nettoie le localStorage et redirige vers /login.

6.4 Routes API utilisées

Fichier	Type	Rôle
GET /directory	LdapList	Récupère la liste des utilisateurs LDAP
GET /api/session	AuthContext	Vérifie la session (optionnel)
POST /auth/refresh-token	client.ts	Rafraîchit le JWT via cookie
GET /logout (PHP)	authApi	Déconnecte l'utilisateur (intranet PHP)

7. Variables d'environnement

7.1 Frontend (.env)

VITE_API_URL	URL de base de l'API Symfony (ex: http://172.x.x.x:8080/api)
VITE_API_URL_LOGOUT	URL de déconnexion PHP (ex: http://172.x.x.x/intranet/logout)
VITE_API_URL_HOME	URL de l'accueil de l'intranet (ex: http://172.x.x.x/intranet)

❏ Important

Ne jamais committer le fichier .env en production. Utiliser .env.example pour partager la structure sans les valeurs.

7.2 Backend (.env)

APP_ENV	dev prod test
CORS_ALLOW_ORIGIN	Regex des origines autorisées (ex: ^https?:/(172\.\20\.\11\.[0-9]+)(:[0-9]+)?\$)
JWT_SECRET_KEY	Clé privée pour signer les JWT
JWT_PUBLIC_KEY	Clé publique pour vérifier les JWT
LDAP_HOST	Adresse du serveur Active Directory
LDAP_PORT	Port LDAP (389 ou 636 pour SSL)

8. Routing

8.1 Configuration des routes

Fichier	Type	Rôle
/	Protégée	Page principale : SearchBar + LdapList
/access-denied	Publique	Page d'accès refusé (403)

8.2 Chargement paresseux (Lazy Loading)

Les pages sont chargées de manière asynchrone via `React.lazy()` pour optimiser le temps de chargement initial. Un composant `Loader` est affiché pendant le chargement.

9. Charte graphique

9.1 Palette de couleurs

#FBBB01 (infranet)	Couleur principale HFF — boutons, accents, bordures
#1F2937 (dark)	Header, thêtes de tableaux, éléments sombres
#F5F5F4 (workspace)	Fond de page principal
#FFFFFF (white)	Fond des composants (cartes, tableaux)
#FFC107 (hover)	Survol des lignes du tableau
#1D6F42 (excel)	Bouton export Excel

9.2 Typographie

Police	Noto Sans Nandinagari (Google Fonts)
Tailles	xs (10px), sm (12px), base (14px)
Style	uppercase + tracking-widest pour les labels

9.3 Composants UI réutilisables

- Button : variants primary, secondary, success, danger, ghost
- Loader : spinner animé avec label optionnel
- Breadcrumb : fil d'ariane en forme de flèche (clipPath)

10. Problèmes connus & solutions

10.1 CORS

Problème

Les requêtes depuis `http://172.20.11.236:5173` vers `http://172.20.11.32` sont bloquées par le navigateur.

Solution

Configurer `NelmioCORSBundle` côté Symfony avec la regex autorisant le réseau `172.20.11.x`.
Vider le cache Symfony après modification : `php bin/console cache:clear`

10.2 Race condition token

Problème

Si le token est lu dans un `useEffect`, le Header peut vérifier l'auth avant que le token soit sauvé, provoquant une déconnexion intempestive.

Solution

Décoder et sauvegarder le token EN DEHORS du composant App (avant le premier rendu React). Le code s'exécute de façon synchrone avant le montage de tout composant.

10.3 `userRole.toUpperCase()` crash

Problème

PHP renvoie roles comme tableau (`["ADMIN"]`), pas comme string. Appeler `.toUpperCase()` sur un tableau provoque une `TypeError`.

Solution

Utiliser `user.roles?.[0]` pour extraire le premier élément, ou `Number(user.role) === 1` pour la détection ADMIN.

11. Lancement du projet

11.1 Prérequis

- Node.js >= 18
- PHP >= 8.1
- Composer
- Accès réseau au serveur Active Directory

11.2 Frontend

```
cd frontend
npm install
cp .env.example .env      # Configurer les variables
npm run dev -- --host      # Développement (accessible sur le réseau)
npm run build              # Production
```

11.3 Backend

```
cd backend
composer install
cp .env.example .env      # Configurer les variables
php bin/console cache:clear
symfony server:start      # ou php -S 0.0.0.0:8080 -t public
```

Documentation rédigée pour les développeurs

09/04/2026